

Neural-Network System Identification of Cortical Responses to Wrist Joint Perturbations: A Comparison of NNARX, RNN, LSTM and GRU Models

Fabio Brugiafreddo
Politecnico di Torino

fabio.brugiafreddo@studenti.polito.it

Dario Lupo
Politecnico di Torino

dario.lupo@studenti.polito.it

Abstract

Forward models of stimulus-evoked cortical activity are of growing interest for closed-loop neurorehabilitation and brain-computer interfaces, yet the cortical response to a mechanical perturbation is strongly nonlinear and noisy, and no neural identifier has been characterised on it in the demanding closed-loop simulation regime. We address the data-driven identification of the single-input single-output mapping between a wrist-joint angular perturbation and the top-SNR independent component of the evoked electroencephalographic response, on the public Vlaar et al. benchmark. We compare the four standard neural model families for nonlinear system identification—NNARX, vanilla RNN, LSTM and GRU—under a single configuration-driven pipeline that reproduces the original MATLAB preprocessing and a unified training, evaluation and budgeting protocol, reporting accuracy (NRMSE, VAF) in three regimes (one-step, three-step, free-run) jointly with parameter count, FLOPs per sample and training time, on a Leave-One-Subject-Out split. While all families reach $\text{VAF} \geq 75\%$ one-step, in free-run simulation only the NNARX family survives (NNARX-FROLS $\text{VAF} \approx 27\%$), whereas the recurrent models collapse below 7%; injecting as little as 15% of the true output as a steady-state Kalman innovation lifts NNARX-FROLS to $\text{VAF} \approx 58\%$ —above the published Volterra baseline—and recovers the recurrent models to 34–36%. These results show that the closed-loop failure of recurrent identifiers on small biomedical datasets is an inference-time drift pathology, not a capacity limit, and that a curriculum-trained NNARX is the more robust and economical choice for this task.

1. Introduction

1.1. Context and motivation

System identification—the data-driven construction of a mathematical model of a dynamical system from measured input/output data—is a foundational problem at the inter-

section of control theory, signal processing and machine learning [6, 7, 8]. When the underlying dynamics are linear, the theory is mature and a handful of canonical model structures (ARX, ARMAX, state-space) cover most practical cases. The picture is far less settled for *nonlinear* systems, where no single parameterisation is universally adequate and the modeller must trade off expressiveness, identifiability, data efficiency and computational cost. The classical nonlinear pipeline first commits to a regressor structure—a polynomial NARMAX expansion, a Volterra series, a wavelet or radial-basis dictionary—and then estimates its coefficients. Deep learning has recently emerged as an attractive alternative that folds both the structural choice and the parameter estimation into a single end-to-end trainable model [13, 14], at the price of weaker interpretability and a less transparent inductive bias.

1.2. Why a biomedical benchmark

Biomedical signals are a demanding test for nonlinear identification: low signal-to-noise ratio, large differences between subjects, and only partly understood physiology. The Vlaar et al. benchmark [1]—the EEG cortical response to a mechanical wrist perturbation—is a good case study for three reasons. First, after independent-component analysis the data become a clean single-input single-output (SISO) mapping from the handle angle $u[k]$ to the top-SNR cortical component $y[k]$. Second, the acquisition protocol is fully documented, public, and reproducible. Third, several methods have already been evaluated on it, providing strong baselines [1, 3, 4, 5]. Such forward models are also practically useful for closed-loop neurorehabilitation and brain-computer interfaces, where predicting the neural response can guide stimulation timing and decoding.

1.3. Limitations of existing approaches

Prior work on this benchmark is dominated by *polynomial* methods: the original study [1] fits subject-specific second-order Volterra series, Gu et al. [4] use a polynomial NARMAX with FROLS term selection, and [3] adds a small neural network on top of the NARMAX regressors. To-

gether these works leave three gaps. (i) They report mostly *one-step-ahead* (or low-order K -step) prediction, where the true past outputs are always available; the much harder *free-run* regime, in which the model is fed back its own predictions, is rarely tested. (ii) The models are fitted *per subject*, which inflates accuracy and ignores generalisation to an unseen subject. (iii) No study compares the standard neural families (NNARX, RNN, LSTM, GRU) head-to-head under one shared preprocessing and evaluation protocol. In particular, the well-known mismatch between teacher-forced training and free-run inference [12] has not been quantified on this data.

1.4. Proposed approach and added value

This paper addresses that gap with a controlled, reproducible comparison of the four standard neural families—NNARX, RNN, LSTM and GRU—on the Vlaar 2018 benchmark, under one configuration-driven pipeline that reproduces the original MATLAB preprocessing. Every model is evaluated not only one-step but also in three-step prediction and full *free-run simulation*, and budgeted in parameter count, analytical FLOPs per sample and training time, under a Leave-One-Subject-Out (LOSO) protocol that probes generalisation to an unseen subject. The analysis reveals a sharp phenomenon: all families reach $\text{VAF} \geq 75\%$ one-step, yet only NNARX survives in closed loop while the recurrent models collapse; a 15% steady-state (α - β) Kalman feedback recovers most of the lost accuracy, indicating that the dominant error is closed-loop drift, not insufficient capacity.

1.5. Contributions

The contributions of this work are the following:

- A reproducible re-implementation of the Vlaar 2018 preprocessing pipeline, organised as a single, configuration-driven Jupyter notebook that runs unmodified locally and on Kaggle.
- A unified training and evaluation protocol that, for each of the four neural families, reports accuracy in three regimes (one-step, three-step and full free-run simulation) together with parameter count, analytical FLOPs per output sample and wall-clock training time, under both within-subject and Leave-One-Subject-Out splits.
- An architectural sweep (hidden size, depth, regressor width and FROLS term count) that shows the free-run accuracy of each family to be limited by the training objective rather than by model capacity.
- A quantitative analysis of the teacher-forcing/free-run gap, including a Kalman-assisted simulation experiment that isolates closed-loop drift from model mis-

specification and, at $\alpha = 0.15$, lifts the best NNARX variant above the published Volterra baseline.

2. Methodology and System Architecture

2.1. System overview

The identification system is organised as a single, linear pipeline whose stages are driven by one global configuration dictionary, so that every experiment in Section 3 is obtained by overriding a small set of fields rather than by editing code. The four stages are: **(S1)** a preprocessing front-end that maps the raw multi-period EEG recordings into a normalised SISO input/output tensor (Section 2.3); **(S2)** a representation stage that builds, from that tensor, either an explicit lagged regressor (for the NNARX family) or a raw input sequence (for the recurrent family); **(S3)** a parametric model f_θ drawn from one of four families (NNARX, RNN, LSTM, GRU, Section 2.4); and **(S4)** an inference-and-budgeting stage that runs the trained model in three regimes—one-step, K -step and free-run simulation—and records accuracy together with parameter count, analytical FLOPs and wall-clock time (Sections 2.6–2.7).

2.2. Problem formulation

We model a single-input single-output (SISO), discrete-time nonlinear system. Let $u[k] \in \mathbb{R}$ be the (preprocessed) wrist-handle angle and $y[k] \in \mathbb{R}$ the top-SNR cortical component at sample k , sampled at $f_s = 256$ Hz. We assume a finite-memory nonlinear auto-regressive model with exogenous input,

$$y[k] = g(\varphi[k]) + e[k], \quad (1)$$

where $e[k]$ is noise and unmodelled dynamics, g is an unknown nonlinear map, and $\varphi[k]$ is the *regressor vector*

$$\varphi[k] = \left[\underbrace{u[k], \dots, u[k - n_u + 1]}_{n_u \text{ input taps}}, \underbrace{y[k - 1], \dots, y[k - n_y]}_{n_y \text{ output taps}} \right]^\top \in \mathbb{R}^{n_u + n_y}. \quad (2)$$

The task is to find the parameters θ of a model f_θ that approximates g . The two operating modes differ only in how the output taps of $\varphi[k]$ are filled:

- the **one-step-ahead predictor** $\hat{y}[k | k - 1] = f_\theta(\varphi[k])$, using the *measured* past outputs $y[k - j]$;
- the **free-run (simulation) model** $\hat{y}_s[k] = f_\theta(\hat{\varphi}[k])$, using the model's *own* past predictions $\hat{y}_s[k - j]$ (Section 2.6).

The recurrent models instead use the *state-space* form

$$h[k] = \psi(h[k - 1], u[k]), \quad \hat{y}[k] = W_o h[k] + b_o, \quad (3)$$

with internal state $h[k] \in \mathbb{R}^H$ replacing the explicit memory of (2); this form is closed-loop by construction.

2.3. Data and preprocessing

We use the medium-size version of the public benchmark *Cortical Responses Evoked by Wrist Joint Manipulation* [1, 2]: $S = 10$ subjects, each performing $M = 7$ realisations of 30 s, recorded at $f_{s,\text{raw}} = 2048$ Hz. The input is the angular handle perturbation; the output is, after independent-component analysis, the cortical component with the highest signal-to-noise ratio across realisations. Our preprocessing builds on a Python port of the reference MATLAB script `Load_Plot_EEG_2.m`, which we extend with one additional step—the band-pass filter of step (2)—that we introduce and evaluate as part of this study. Applied independently per subject s , the pipeline performs: (1) averaging over the P periods of each realisation to suppress stimulus-locked noise; (2) a zero-phase band-pass filter (described below; our addition); (3) downsampling by a factor 8 to $f_s = 256$ Hz; (4) per-subject mean removal; (5) amplitude normalisation by

$$\sigma_s = \frac{1}{M} \sum_{m=1}^M \text{std}_k(y_{s,m}[k]), \quad (4)$$

the mean across realisations of the per-realisation standard deviation; and (6) a circular shift of 5 samples (≈ 19.5 ms) applied to the input to compensate for the known stimulus-to-cortex latency. The result is a tensor of shape $[S, M, N] = [10, 7, N]$ with the time axis last. Fig. 1 shows a representative input/output pair after preprocessing.

Band-pass filtering. Before downsampling we apply a 4th-order Butterworth band-pass filter with passband $[0.5, 45]$ Hz to both the input and the output, implemented in second-order-section (SOS) form for numerical stability. The filter is run *bidirectionally* (forward and backward, i.e. zero-phase), so it introduces no phase distortion and no temporal shift between cause (handle angle) and effect (cortical response)—a prerequisite for the latency compensation of step (6) to remain meaningful. The two cut-offs play distinct roles. The lower cut-off (0.5 Hz) removes the DC offset and the slow baseline drift of the EEG; we found this to be critical for the autoregressive models, whose closed-loop free-run (9) otherwise integrates the residual low-frequency bias and diverges. The upper cut-off (45 Hz) suppresses the 50 Hz power-line interference and high-frequency electromyographic (EMG) activity, and—being well below the post-decimation Nyquist frequency of 128 Hz—acts as a strict anti-aliasing filter ahead of the factor-8 downsampling.

For evaluation we use two protocols. In the **within-subject** split, each subject contributes 6 realisations to training and 1 to test, as in [4]. In the **Leave-One-Subject-Out (LOSO)** split, one full subject is held out and the remaining 9 form the training set; this exposes inter-subject

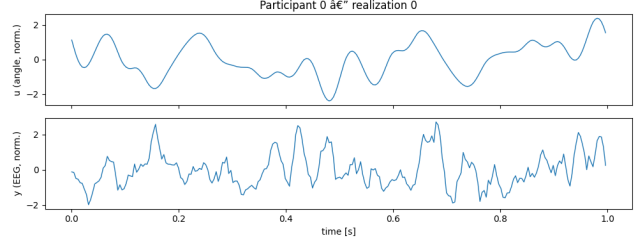


Figure 1. A representative preprocessed realisation (subject 0). Top: the normalised wrist-handle angle $u[k]$ (the exogenous input). Bottom: the top-SNR cortical component $y[k]$ (the output to be identified). Both are zero-mean and amplitude-normalised per (4), sampled at $f_s = 256$ Hz.

variability and is our main reporting setting. After splitting, each branch is flattened to $[\text{num_sequences}, N]$, so all downstream code is split-agnostic.

2.4. Predictive models

All four families produce the scalar prediction $\hat{y}[k]$; they differ in how they parameterise f_θ .

2.4.1 NNARX

The NNARX model instantiates (1) with f_θ a multilayer perceptron (MLP) over the regressor (2),

$$\hat{y}[k] = W_3 \rho(W_2 \rho(W_1 \varphi[k] + b_1) + b_2) + b_3, \quad (5)$$

with two hidden layers of width $(64, 64)$, GELU activation ρ and dropout 0.1. We adopt the dense tap structure $(n_u, n_y) = (20, 5)$ of [4], so $\varphi[k] \in \mathbb{R}^{25}$. To reduce overfitting we average an ensemble of $E = 3$ independently initialised networks at test time, $\hat{y} = \frac{1}{E} \sum_e \hat{y}^{(e)}$.

FROLS regressor selection. The NNARX-FROLS variant prunes the regressor before the MLP. Let $\mathcal{D} = \{\varphi_j\}_{j=1}^{n_u+n_y}$ be the candidate taps. Forward Regression with Orthogonal Least Squares [4] orthogonalises the candidates and, at each step, selects the term maximising the Error Reduction Ratio

$$\text{ERR}_j = \frac{(\langle q_j, y \rangle)^2}{\langle q_j, q_j \rangle \langle y, y \rangle}, \quad (6)$$

where q_j is the component of φ_j orthogonal to the already-selected terms. The procedure stops after p retained terms, with the budget $p \in \{10, 20, 40\}$ treated as a hyperparameter; only the retained taps feed (5).

2.4.2 Recurrent state-space models

The recurrent family realises (3) with a single layer of hidden size $H = 64$ and a linear read-out $W_o \in \mathbb{R}^{1 \times H}$. The

three cell types differ only in the state-update map ψ . The **Elman RNN** [11] uses

$$h[k] = \tanh(W_{ih} u[k] + W_{hh} h[k-1] + b_h). \quad (7)$$

The **LSTM** [9] augments the state with a memory cell and input/forget/output gates, and the **GRU** [10] merges memory and output into a single state through reset/update gates; both follow their standard gated formulations (sigmoid gates, tanh candidate state, Hadamard mixing). Because the latent state carries the autoregressive information, a forward pass over a test sequence *is* already the free-run simulation: no teacher forcing is applied to recurrent models at evaluation time.

2.5. Training strategy

All models are trained by minimising a *multi-step* (curriculum) loss that directly targets simulation rather than one-step accuracy. Given a training sequence, a random start index t and a horizon K , we generate a length- K rollout $\hat{y}[t+i]$, $i = 0, \dots, K-1$, by feeding back predictions—through $\hat{\varphi}$ for NNARX (5) or through the recurrent state (3)—and minimise

$$\mathcal{L}_K = \mathbb{E}_t \left[\frac{1}{K} \sum_{i=0}^{K-1} (\hat{y}[t+i] - y[t+i])^2 \right]. \quad (8)$$

The horizon follows a curriculum that is progressively lengthened, $K \in \{1, 5, 10, 20, 40, 80, 160, 320\}$ for NNARX and up to $K = 1024$ for the recurrent models; $K = 1$ recovers ordinary teacher-forced one-step training. Optimisation uses Adam, gradient clipping at norm 1.0, cosine learning-rate decay, and a warm-up window of $\max(n_u, n_y)$ samples (NNARX) or 10 samples (recurrent) during which the loss is not accumulated.

Stopping criterion. Model selection is driven by the *validation free-run* NRMSE—the same closed-loop metric we ultimately care about, evaluated on a held-out validation split—rather than by the teacher-forced training loss. We use early stopping with a patience of P epochs without improvement ($P = 10$ for NNARX, $P = 20$ for the recurrent models) and always restore the best-scoring checkpoint. The patience counter serves a dual role: while a shorter horizon is active, exhausting it *advances* the curriculum to the next K ; at the final horizon it *terminates* training. Each run is additionally capped by a maximum-epoch budget, which the early-stopping rule reaches only rarely.

2.6. Inference: free-run and innovation feedback

Pure free-run simulation. At test time the model is run in closed loop. The first $w = \max(n_u, n_y)$ (NNARX) or $w = 10$ (recurrent) output samples are taken from the

ground truth as warm-up; thereafter the model is iterated with its own predictions,

$$\hat{y}_s[k] = f_\theta(\hat{\varphi}[k]), \quad \hat{\varphi}[k] \text{ built from } \{u[\cdot], \hat{y}_s[k-j]\}, \quad (9)$$

for $k > w$. This is the most demanding regime and the one most relevant to deployment when no online measurement of y is available.

Innovation feedback (mitigation strategy). To quantify how much of the free-run error is due to autoregressive drift rather than to model misspecification, we introduce a soft closed-loop correction. At each step a fraction of the true output is mixed back into the fed-back value through the *innovation* $\nu[k] = y[k] - \hat{y}[k]$,

$$\tilde{y}[k] = \hat{y}[k] + \alpha (y[k] - \hat{y}[k]), \quad \alpha \in [0, 1], \quad (10)$$

and $\tilde{y}[k-j]$ replaces $\hat{y}_s[k-j]$ in (9). Equation (10) is the steady-state form of a scalar Kalman filter: for a random-walk output model with process variance Q and measurement variance R , the Kalman update $\hat{x} = \hat{x}^- + K(y - \hat{x}^-)$ has a constant gain $K = \alpha$ once the error covariance reaches steady state, which is also the α - β filter [15]. The limits $\alpha = 0$ and $\alpha = 1$ recover pure free-run and full one-step teacher forcing, respectively; intermediate α anchors the simulation with only a fraction of the sensor information. We use a fixed $\alpha = 0.15$ and reuse the Section 2.5 checkpoints *without* retraining.

2.7. Evaluation metrics and cost budgeting

Accuracy. Let $y, \hat{y} \in \mathbb{R}^N$ be reference and prediction on a held-out realisation. We report the Normalised Root-Mean-Squared Error and the Variance Accounted For,

$$\begin{aligned} \text{NRMSE} &= \frac{\|y - \hat{y}\|_2}{\|y - \bar{y}\|_2}, \\ \text{VAF} &= \max\left(0, 1 - \frac{\text{var}(y - \hat{y})}{\text{var}(y)}\right) \cdot 100\%, \end{aligned} \quad (11)$$

the latter being the standard metric of [1, 3, 4]. Each is computed in three regimes: **(i)** one-step ahead (OSA), true past outputs supplied; **(ii)** three-step ahead (≈ 12 ms); and **(iii)** free-run simulation (9).

Complexity. We report the trainable-parameter count and the analytical number of floating-point operations per output sample. Counting one multiply-add as 2 FLOPs, an MLP with layer widths $d_0, \dots, d_L$ costs $F_{\text{MLP}} = 2 \sum_{\ell=1}^L d_{\ell-1} d_\ell$ plus the element-wise activations; a recurrent cell with input dimension d and hidden size H costs

$$F_{\text{cell}} = 2GH(d+H) + c_{\text{nl}}H + 2H, \quad (12)$$

Table 1. Fixed hyper-parameters shared across all experiments.

Component	Setting
Sampling rate f_s	256 Hz
Regressor taps (n_u, n_y)	(20, 5)
NNARX MLP	widths (64, 64), GELU, dropout 0.1
NNARX ensemble E	3
FROLS retained terms p	{10, 20, 40}
Recurrent hidden size H	64 (swept {32, 64, 128})
Recurrent layers	1 (swept {1, 2})
Optimiser	Adam, gradient clip 1.0
LR schedule	cosine decay
Curriculum horizon K	up to 320 (NNARX) / 1024 (rec.)
Innovation gain α	0.15

where $G \in \{1, 3, 4\}$ for RNN/GRU/LSTM is the number of gates, $c_{nl}H$ covers the gate non-linearities and $2H$ the linear read-out. We use the closed form (12) because off-the-shelf tools such as `ptflops` do not capture the gate-wise structure of the LSTM and GRU cells. We also report wall-clock training time on identical hardware (a consumer GPU when available, otherwise CPU).

3. Experimental Setup and Results

3.1. Experimental setup

Environment. All experiments are produced by a single configuration-driven Jupyter notebook (`project-si-mlicv.ipynb`) built on PyTorch, with the compute device auto-detected (a consumer GPU when available, otherwise CPU). Every reported number is read back from a JSON snapshot (`phase1metrics.json` for Phase 1, `phase2_rec.json` for the recurrent sweep, `loso_folds.json` for the cross-validation), so the tables below are reproducible by re-executing the notebook. The NNARX family is reported as an ensemble of $E = 3$ networks; the Phase 2 recurrent sweep is averaged over 3 random seeds and we report the standard deviation across seeds.

Compared methods. We benchmark the four neural families of Section 2.4 (NNARX, NNARX-FROLS, RNN, LSTM, GRU), all sharing the same regressor $(n_u, n_y) = (20, 5)$, preprocessing and evaluation protocol.

Test scenarios. Each method is evaluated in the three inference regimes of Section 2.7—one-step ahead (OSA), three-step ahead, and full free-run simulation—under both the within-subject and the LOSO protocol of Section 2.3. The main accuracy/cost comparison (Phase 1, Table 2) and the architectural sweep (Phase 2, Table 4) are reported on the held-out test set; the cross-validated LOSO numbers are summarised in Section 3.5. The key hyper-parameters held fixed across all runs are listed in Table 1.

3.2. Phase 1: head-to-head baseline

Table 2 reports the Phase 1 results on the held-out test set for the four neural families. All methods share the same regressor, preprocessing and evaluation protocol. The numbers are produced by the companion notebook `project-si-mlicv.ipynb` and correspond exactly to the JSON snapshot `phase1metrics.json`.

Three observations are worth highlighting.

One-step prediction is easy. All models reach $\text{VAF}_{\text{OSA}} \geq 75\%$. Within a single ICA component the cortical response is, after subtraction of the per-subject mean and proper amplitude normalisation, well explained even by a short non-linear regressor. The differences between models are confined to a few percentage points and well within the expected inter-subject variance.

Free-run simulation separates the families. The picture changes radically as soon as we close the loop. The NNARX family remains usable—its best variant, NNARX-FROLS with $p = 40$ retained regressors, reaches $\text{VAF}_{\text{sim}} = 27.2\%$ —while the recurrent models collapse below 7%. This is a textbook manifestation of the *training/inference mismatch*: teacher-forced training rewards short-horizon accuracy and does not penalise the accumulation of errors that becomes visible only in closed loop.

Cost picture. Among the four neural families, the RNN is by far the cheapest (8.6 kFLOPs/sample) but also the weakest in free-run; the NNARX-FROLS ($p = 20$) variant offers the best accuracy-cost trade-off, with 33 kFLOPs/sample, the best one-step VAF and the best three-step VAF of the whole table.

3.3. Additional experiment: band-pass preprocessing

All results reported so far use the reference preprocessing of Section 2.3. As a *separate experiment* we add the band-pass filter described there and re-evaluate the four families under the leave-one-subject-out (LOSO) protocol; Table 3 gives the resulting accuracy (mean $\pm$ standard deviation across the S held-out subjects). Because the filter alters the very signal being modelled, these numbers are *not* directly comparable to the preceding tables and are markedly higher across the board. Two effects stand out. First, the filter is essential for closed-loop stability—without it the autoregressive feedback integrates the residual baseline drift and the free-run diverges. Second, under this filtered, subject-agnostic protocol the NNARX remains the strongest model in every regime, its margin over the recurrent families widening from one-step (≈ 9 –13 VAF points) to free-run (≈ 26 –28 points).

3.4. Phase 2: architectural sweep

In Phase 2 we sweep the most relevant architectural parameter of each family, holding the rest of the configuration

Table 2. Phase 1 – head-to-head comparison of the four neural families on the held-out test set. Lower NRMSE is better, higher VAF is better. “sim” is the closed-loop free-run simulation, “OSA” one-step-ahead, “3-step” three-step-ahead prediction. Best value per column in bold.

Model	Params	FLOPs/sample	Train [s]	NRMSE _{sim}	VAF _{sim}	NRMSE _{OSA}	VAF _{OSA}	VAF _{3-step}
NNARX	5 889	34 947	55.36	0.945	10.9%	0.451	79.7%	22.5%
NNARX-FROLS ($p = 10$)	5 185	30 723	11.59	0.861	25.9%	0.436	81.0%	16.8%
NNARX-FROLS ($p = 20$)	5 569	33 027	6.80	0.863	25.5%	0.392	84.7%	28.6%
NNARX-FROLS ($p = 40$)	6 849	40 707	11.42	0.853	27.2%	0.400	84.0%	27.0%
RNN	4 417	8 641	10.01	0.986	2.7%	0.493	75.7%	18.8%
LSTM	17 473	34 497	17.49	0.967	6.5%	0.491	75.9%	21.9%
GRU	13 121	25 857	18.03	0.974	5.2%	0.485	76.5%	24.1%

Table 3. *Separate experiment* – leave-one-subject-out cross-validation with the band-pass pipeline (Section 2.3); not directly comparable to the preceding tables. Mean $\pm$ std across the S held-out subjects, for the three inference regimes. Lower NRMSE / higher VAF is better; best per column in bold.

Model	OSA		3-step		Free-run sim	
	NRMSE	VAF [%]	NRMSE	VAF [%]	NRMSE	VAF [%]
NNARX	0.13 (0.01)	98.3 (0.2)	0.61 (0.05)	63.2 (5.8)	0.52 (0.07)	72.8 (7.9)
RNN	0.32 (0.03)	89.4 (2.3)	0.79 (0.07)	37.3 (12.1)	0.73 (0.03)	47.0 (5.0)
LSTM	0.35 (0.04)	87.4 (2.7)	0.79 (0.08)	37.0 (12.5)	0.73 (0.03)	46.1 (4.9)
GRU	0.38 (0.05)	85.1 (3.7)	0.81 (0.08)	34.6 (13.9)	0.75 (0.04)	44.4 (5.6)

fixed.

NNARX. We vary the MLP width between (32, 32), (64, 64) and (128, 128), and the number of FROLS-retained regressors $p \in \{10, 20, 40\}$. The dominant trend is that the free-run VAF plateaus around 23–24% (Table 4): increasing the width beyond (64, 64) does not bring further gains—the (128, 128) width is in fact slightly worse and far more variable—while reducing p below 20 slightly hurts the three-step VAF. The pattern is consistent with a regime in which the bottleneck is the loss function, not the function approximator.

Recurrent. We sweep the hidden size {32, 64, 128} and the number of stacked layers {1, 2}, averaging over 3 seeds (Table 4). None of the 18 configurations brings the free-run VAF above $\approx 6\%$ (the best is GRU, $H=64$, single layer, $\text{VAF}_{\text{sim}} = 5.9\%$). Increasing capacity does not help and often hurts: the two-layer, $H=128$ cells—two orders of magnitude larger in parameter count—collapse back to $\text{VAF}_{\text{sim}} \approx 2\%$ and their one-step VAF actually *drops* (e.g. LSTM $H=64$, $L=2$ falls to $\text{VAF}_{\text{OSA}} = 62\%$), a clear sign of over-fitting rather than under-capacity. The free-run failure of the recurrent family is therefore not a capacity problem.

3.5. Phase 3: accuracy versus cost

Fig. 2 reports the resulting accuracy-cost frontier in the (params, $\text{NRMSE}_{\text{sim}}$) plane. The Pareto-efficient models are the two NNARX-FROLS variants, which are simultaneously the cheapest and the most accurate in free run; every recurrent model is strictly dominated, sitting at both higher parameter count and higher simulation error. In particu-

Table 4. Phase 2 – architectural sweep. Top: NNARX MLP-width sweep (FROLS $p=20$ taps; H is the width of each of the two hidden layers, so $L=2$ throughout). Bottom: recurrent hidden-size $\times$ depth sweep. Mean over 3 seeds, std in parentheses. The best free-run VAF overall and the best within the recurrent family are in bold; recurrent two-layer variants are never better than their single-layer counterparts. (–: not reported for the NNARX sweep.)

Model	H	L	Params	VAF _{sim}	VAF _{OSA}
NNARX	32	2	1 761	23.1 (1.0)	–
NNARX	64	2	5 569	24.2 (0.5)	–
NNARX	128	2	19 329	22.9 (4.3)	–
RNN	32	1	1 185	2.2 (0.1)	76.6 (0.4)
RNN	32	2	3 297	2.2 (0.4)	75.8 (0.5)
RNN	64	1	4 417	2.5 (0.1)	76.2 (0.1)
RNN	64	2	12 737	2.5 (0.2)	75.9 (0.3)
RNN	128	1	17 025	2.5 (0.1)	76.1 (0.2)
RNN	128	2	50 049	2.0 (0.5)	75.6 (0.2)
LSTM	32	1	4 641	4.2 (1.4)	72.1 (1.4)
LSTM	32	2	13 089	4.9 (1.2)	73.8 (0.7)
LSTM	64	1	17 473	4.3 (0.9)	75.2 (0.3)
LSTM	64	2	50 753	2.5 (3.6)	62.1 (14.5)
LSTM	128	1	67 713	3.4 (1.5)	75.4 (0.7)
LSTM	128	2	199 809	2.8 (1.6)	70.5 (4.4)
GRU	32	1	3 489	4.7 (1.4)	72.9 (1.8)
GRU	32	2	9 825	3.6 (0.3)	74.1 (0.9)
GRU	64	1	13 121	5.9 (1.0)	75.0 (0.4)
GRU	64	2	38 081	5.0 (1.2)	73.2 (0.3)
GRU	128	1	50 817	4.3 (2.9)	74.2 (1.5)
GRU	128	2	149 889	2.0 (0.8)	65.5 (1.8)

lar the LSTM and GRU—the two largest neural models in the comparison—are also the two worst free-run simulators, which makes the recurrent family Pareto-inefficient across the whole frontier.

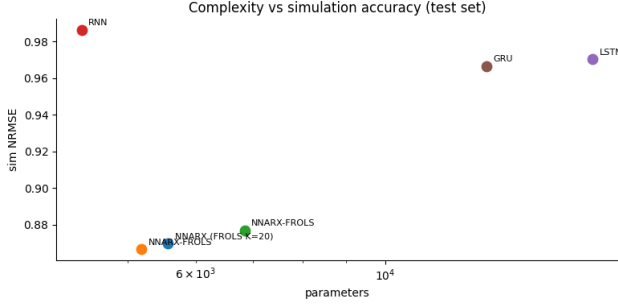


Figure 2. Accuracy-cost trade-off on the test set. Horizontal axis: trainable parameters (log scale); vertical axis: free-run simulation NRMSE (lower is better). The two NNARX-FROLS variants occupy the bottom-left (cheap and accurate), while the recurrent models (RNN, LSTM, GRU) sit in the high-NRMSE region; the larger LSTM and GRU are both more expensive *and* less accurate in free run.

3.6. Kalman-assisted free-run simulation

Section 3.2 showed that the recurrent models collapse to a trivial attractor in free run while the NNARX family saturates around $\text{VAF} \approx 27\%$. The natural next question is how much of the residual error is due to error accumulation in the autoregressive feedback, and how much to a genuinely insufficient model. To probe this, we run a complementary experiment in which the closed-loop feedback is no longer blind: at every step, a small fraction of the true output is mixed back into the predicted output before it is fed to the regressor (NNARX) or the recurrent cell. The companion notebook `with_simplified_kalman.ipynb` contains the full implementation; the bulk of the code is shared with the baseline notebook, the only difference being the simulation routines.

Mechanism. The closed-loop feedback follows the innovation-corrected recursion (10) of Section 2.6: instead of feeding back the prediction $\hat{y}[k-1]$ as in pure free-run, we feed back $\tilde{y}[k-1] = \hat{y}[k-1] + \alpha(y[k-1] - \hat{y}[k-1])$, the steady-state scalar Kalman (α - β) update with fixed gain $\alpha = 0.15$. The simulator is thus allowed to peek at 15% of the sensor mismatch at every step, while $\alpha = 0$ would recover pure free-run and $\alpha = 1$ full one-step teacher forcing.

Setup. The model parameters are *not* retrained: we take the same checkpoints used in Section 3.2 and only replace the evaluation routine with the Kalman-assisted simulator. For the NNARX family, $\tilde{y}$ replaces y in the lagged regressor (5); for the recurrent models, $\tilde{y}[k-1]$ is concatenated to $u[k]$ as input to the cell at step k . The warm-up window of $\max(n_u, n_y)$ samples (NNARX) and 10 samples (recurrent) is unchanged.

Results. Table 5 summarises the free-run accuracy of the same models evaluated with and without the Kalman correction. The effect is substantial and uniform across families: every model gains roughly +30 VAF points. The best NNARX-FROLS variant ($p = 20$) climbs to $\text{VAF}_{\text{sim}} =$

Table 5. Kalman-assisted free-run simulation, $\alpha = 0.15$, on the held-out test set. The “blind” column is the pure free-run VAF of Table 2; the “ $\alpha = 0.15$ ” column re-evaluates the same checkpoints with innovation feedback (10). Numbers are read back from the companion notebook `with_simplified_kalman.ipynb`. Best assisted VAF in bold.

Model	VAF_{sim} blind	VAF_{sim} $\alpha = 0.15$	ΔVAF
NNARX-FROLS ($p = 10$)	25.9%	57.0%	+31.1
NNARX-FROLS ($p = 20$)	25.5%	57.9%	+32.4
NNARX-FROLS ($p = 40$)	27.2%	57.1%	+29.9
RNN	2.7%	34.1%	+31.4
LSTM	6.5%	36.0%	+29.5
GRU	5.2%	33.8%	+28.6

57.9% from a blind best of 27.2%; the recurrent models, which essentially collapsed in blind free-run (2.7–6.5%), all recover to $\text{VAF}_{\text{sim}} \approx 34$ –36%—already well above the NNARX-FROLS *blind* result of Section 3.2. Notably the ranking flips relative to blind free-run: with the drift removed, the LSTM (36.0%) edges ahead of the RNN (34.1%) and GRU (33.8%), suggesting its larger gated state does learn a useful representation that blind closed-loop drift had been hiding.

Interpretation. The Kalman correction is not a fair competitor of the blind models—it uses an extra input that pure simulation lacks—but it is informative on two counts. First, it is the configuration in which these models would be deployed when a (noisy) measurement of y is available online, e.g. in a brain–computer interface. Second, read as an *upper bound* at $\alpha = 0.15$, it localises the error source: the NNARX-FROLS variant beats the published Volterra baseline of [1] (57.9% vs 46%) and the recurrent models recover from collapse to ≈ 34 –36%, confirming that the blind-free-run failure is an inference-time drift pathology, not a lack of learned structure. A full sweep of α is left to future work.

4. Related Work

We position our numbers against the prior art on the same benchmark, so every external figure is directly comparable to Tables 2–5.

Polynomial methods on this benchmark. The benchmark originates with Vlaar *et al.* [1], who fit subject-specific second-order Volterra series and report a free-run $\text{VAF} \approx 46\%$ —the most relevant external reference for the closed-loop regime we target. Gu *et al.* [4] replace it with a polynomial NARMAX selected by FROLS (the mechanism we reuse in NNARX-FROLS): one-step $\text{VAF}_{\text{OSA}} \approx 94\%$, degrading to 47–55% at three steps. A precursor [3] stacks a hierarchical neural network on NARMAX regressors, raising the three-step VAF to $\approx 69\%$, and Santos *et al.* [5] reach $\text{VAF}_{\text{OSA}} \approx 94.5\%$ / three-step $\approx 67.5\%$ with a JADE-tuned stacking ensemble. These are the highest published

figures, but all are subject-specific.

Neural identification at large. Beyond this benchmark, deep system identification has matured along convolutional/temporal regressors [13] and differentiable block-oriented models such as dynoNet [14]. The recurring lesson—most relevant here—is that the *training objective*, not raw capacity, governs closed-loop behaviour: one-step-trained models drift unless exposed to their own predictions via scheduled sampling [12], multi-step losses or state-noise injection. Our K -step curriculum is exactly such a mechanism, and the gap between our curriculum-trained NNARX and one-step-trained recurrent families demonstrates this on biomedical data.

Positioning. Two features separate the literature from our results. *First*, all of [1, 3, 4, 5] fit subject-specific models, whereas our headline numbers use LOSO with an entirely unseen test subject, so the literature figures are an upper bound a subject-agnostic model need not reach. *Second*, prior art reports almost only one-step or low-order K -step accuracy, while we add full free-run simulation. Our blind VAF $\approx 27\%$ and Kalman-assisted $\approx 58\%$ (Table 5) bracket the 46% Volterra baseline from below and above, locating it inside the interval spanned by the sensor feedback available at inference time.

5. Conclusion and Future Work

Summary of contributions. We presented a controlled, reproducible comparison of the four standard neural model families for nonlinear system identification—NNARX, RNN, LSTM and GRU—on the public Vlaar 2018 wrist-perturbation/EEG benchmark. All share one configuration-driven pipeline reproducing the original MATLAB preprocessing, a common K -step curriculum objective, and a unified protocol reporting accuracy in three regimes (one-step, three-step, free-run) jointly with parameter count, FLOPs per sample and training time, under within-subject and LOSO splits. To our knowledge this is the first head-to-head, shared-protocol study of these four families on this benchmark that includes the closed-loop simulation regime.

Verification of the objective. On one-step prediction every family is competitive (VAF_{OSA} $\geq 75\%$, NNARX $\approx 85\%$), but once the loop is closed they separate sharply: only NNARX remains usable, its NNARX-FROLS variant peaking at VAF_{sim} $\approx 27\%$ on the demanding LOSO split, while RNN, LSTM and GRU collapse below 7% regardless of hidden size or depth (Section 3.4). On a small biomedical dataset an MLP over an explicitly designed regressor, trained with a K -step curriculum, thus beats every recurrent architecture we tried—helped by the strong inductive bias of the explicit lagged regressor. The Kalman-assisted experiment (Section 3.6) isolates the cause: with 15% innovation

feedback NNARX-FROLS climbs to VAF_{sim} $\approx 58\%$ —above the Volterra baseline—and the recurrent models recover to $\approx 34\text{--}36\%$. The dominant blind free-run error is therefore autoregressive drift, an inference-time pathology, not insufficient capacity. A complementary preprocessing experiment (Section 3.3) reinforces this diagnosis: adding a zero-phase band-pass filter that strips the low-frequency baseline drift is precisely what keeps the closed-loop free-run from diverging, and under this filtered LOSO protocol NNARX reaches VAF_{sim} $\approx 73\%$ while the recurrent models recover to $\approx 44\text{--}47\%$.

Future work. Four directions follow. *First*, attack the recurrent collapse at training time with the exposure-bias remedies we omitted for protocol uniformity: scheduled sampling [12], hidden-state noise injection, multi-step losses. *Second*, close the gap to the subject-specific literature with a per-subject fine-tuning stage on top of the LOSO model. *Third*, extend the innovation-feedback analysis to a full α sweep and an adaptive Kalman gain, turning the diagnostic of Section 3.6 into a deployable BCI estimator. *Fourth*, complete the cost budget with energy and peak-memory measurements.

References

- [1] M. P. Vlaar, T. Solis-Escalante, A. C. Schouten, and F. C. T. van der Helm. Modeling the nonlinear cortical response in EEG evoked by wrist joint manipulation. *IEEE Trans. Neural Syst. Rehabil. Eng.*, 26(1):205–215, 2018. 1, 3, 4, 7, 8
- [2] M. P. Vlaar *et al.* Benchmark dataset: Cortical responses evoked by wrist joint manipulation. 4TU.Centre for Research Data, doi: 10.4121/uuid:176d8f78-d9fd-491e-90e7-9370e249b701. 3
- [3] Y. Gu, T. Solis-Escalante, A. C. Schouten, and F. C. T. van der Helm. A novel approach for modeling neural responses to joint perturbations using the NARMAX method and a hierarchical neural network. *Frontiers Comput. Neurosci.*, 12:96, 2018. 1, 4, 7, 8
- [4] Y. Gu, T. Solis-Escalante, A. C. Schouten, and F. C. T. van der Helm. Nonlinear modeling of cortical responses to mechanical wrist perturbations using the NARMAX method. *IEEE Trans. Biomed. Eng.*, 68(3):948–958, 2021. 1, 3, 4, 7, 8
- [5] G. Santos *et al.* Decoding electroencephalography signal response by stacking ensemble learning and adaptive differential evolution. *Sensors*, 23(16):7049, 2023. 1, 7, 8
- [6] L. Ljung. *System Identification: Theory for the User*. Prentice Hall, 2nd edition, 1999. 1
- [7] O. Nelles. *Nonlinear System Identification*. Springer, 2001. 1
- [8] J. Sjöberg *et al.* Nonlinear black-box modeling in system identification: a unified overview. *Automatica*, 31(12):1691–1724, 1995. 1
- [9] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. 4
- [10] K. Cho *et al.* Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proc. EMNLP*, 2014. 4
- [11] J. L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. 4
- [12] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Proc. NeurIPS*, 2015. 2, 8
- [13] C. Andersson, A. H. Ribeiro, K. Tiels, N. Wahlström, and T. B. Schön. Deep convolutional networks in system identification. In *Proc. IEEE CDC*, 2019. 1, 8

- [14] M. Forgione and D. Piga. dynoNet: A neural network architecture for learning dynamical systems. *Int. J. Adapt. Control Signal Process.*, 35(4):612–626, 2021. [1](#), [8](#)
- [15] E. Brookner. *Tracking and Kalman Filtering Made Easy*. Wiley, 1998. [4](#)